Types, UTF-8, Iota et Contrôle

---

1. Variables et Système de Types

Go est un langage à **typage statique, fort et explicite**. Contrairement au C, il n'existe **aucune promotion ou conversion implicite** entre les types numériques, même si aucune perte de précision n'est possible : additionner un `int32` et un `int64` sans conversion explicite provoque une erreur de compilation immédiate.

1.1 Le concept fondamental de "Zero Value"

En C ou C++, déclarer une variable locale sur la pile (`int x;`) sans l'initialiser conserve les bits résiduels présents à cette adresse mémoire (*undefined behavior* / mémoire poubelle).

En Go, **toute allocation mémoire est garantie d'être mise à zéro** par le runtime avant utilisation. Chaque type possède sa *Zero Value* :

Type	Zero Value	Comportement / Équivalent C
Entiers (`int`, `uint8`, `int64`...)	`0`	Tous les bits à `0`.
Flottants (`float32`, `float64`)	`0.0`	Tous les bits à `0` (norme IEEE-754).
Booléen (`bool`)	`false`	`0` sur 1 octet.
Chaîne (`string`)	`""`	Chaîne vide valide (et **non** un pointeur `NULL` !).
Pointeurs (`*T`)	`nil`	Équivalent de `NULL` en C.
Types références (`slice`, `map`, `chan`, `func`, `interface`)	`nil`	Descripteur interne mis à zéro.

1.2 Les syntaxes de déclaration et conventions de style

Go propose différentes syntaxes selon la portée et l'intention du développeur :

```go
package main
```

```go
// 1. Hors fonction (niveau package) : 'var' est obligatoire (':=' est interdit)
var VersionApplication = "1.0.0"

func exempleDeclarations() {
	// 2. 'var' sans initialisation : indique clairement qu'on souhaite partir de la Zero Value (0)
	var compteurErreurs int

	// 3. Déclaration courte ':=' (Inférence de type) : la norme à l'intérieur des fonctions
	port := 8080          // inféré en 'int' (64 bits sur arch 64 bits)
	timeout := 5.5        // inféré en 'float64'
	estActif := true      // inféré en 'bool'

	// 4. 'var' avec type explicite : quand on veut un type spécifique différent du type inféré par défaut
	var codeStatut uint16 = 200 // avec ':=', 200 aurait été typé en 'int'

	// 5. Déclaration groupée (factorisation propre)
	var (
		hote   = "127.0.0.1"
		maxCon = 1024
	)

	// Conversion explicite obligatoire entre types numériques :
	var total int64 = int64(port) + int64(codeStatut)

	_, _, _, _, _, _ = compteurErreurs, timeout, estActif, hote, maxCon, total
}
```

1.3 Les types numériques et l'opérateur binaire `&^` (Bit Clear)

Go fournit des types à taille garantie (`int8`, `int16`, `int32`, `int64` et leurs équivalents non signés `uint8`, `uint16`, `uint32`, `uint64`) ainsi que les types dépendants de l'architecture CPU (`int`, `uint`, `uintptr`).

En plus des opérateurs binaires hérités du C (`&` ET, `|` OU, `^` XOR, `<<` et `>>` décalages), Go introduit un opérateur spécifique très utilisé en programmation système : **`&^` (AND NOT / Bit Clear)**.

- En C, pour éteindre un bit précis dans un masque, on écrit : `masque = masque & ~FLAG;`
- En Go (où `^x` sert à la fois au XOR binaire `a ^ b` et à la négation bit à bit unaire `^a`), on utilise directement `masque &^= FLAG` (qui signifie : *"garde les bits de `masque`, sauf ceux qui sont à 1 dans `FLAG`"*).

---

2. Chaînes de caractères (`string`), `byte` et `rune`

2.1 Architecture mémoire d'une `string` en Go

En C, une chaîne `char*` est une suite d'octets terminée par le caractère nul `\0` (calculer `strlen` coûte $O(n)$ et le dépassement de tampon est fréquent).

En Go, une `string` est une structure interne de **16 octets** (sur architecture 64 bits) composée de deux champs :

1. **`Data` (8 octets) :** Un pointeur vers le tableau d'octets sous-jacent (alloué en zone mémoire en lecture seule).
2. **`Len` (8 octets) :** Le nombre exact d'octets de la chaîne.

Conséquences directes :

- Appeler `len(maChaine)` ne parcourt pas la chaîne : c'est une lecture directe du champ `Len` en $O(1)$.
- Passer une `string` de 50 Mo à une fonction ne copie pas les 50 Mo : Go ne copie que le descripteur de 16 octets !
- Les `string` sont **strictement immuables** : tenter de modifier `maChaine[0] = 'X'` est interdit à la compilation.

2.2 Encodage UTF-8 : `byte` vs `rune`

Go utilise nativement l'encodage **UTF-8** (conçu par ailleurs par Ken Thompson et Rob Pike, les créateurs de Go). En UTF-8 :

- Les caractères ASCII standards (codes `0` à `127`) occupent **1 seul octet**.
- Les caractères accentués européens (é, à, ç) occupent **2 octets**.
- Les caractères asiatiques ou symboles spéciaux (€, ☕) occupent **3 octets**.
- Les émojis (🚀, 🔥) occupent **4 octets**.

Pour manipuler cela sans ambiguïté, Go définit deux alias de types dans son langage :

- **`byte`** est un alias strict de **`uint8`** : il représente **1 octet brut**.
- **`rune`** est un alias strict de **`int32`** : il représente **1 point de code Unicode complet** (1 caractère logique, qu'il fasse 1, 2, 3 ou 4 octets en mémoire).

```
Go
texte := "Go 🚀"
```

```
fmt.Println(len(texte)) // Affiche 7 (octets) et NON 4 (caractères) ! ('G'=1,
'o'=1, ' '=1, '🚀'=4)

// 1. Parcours par index (for i := 0; i < len(texte); i++) -> Lit octet par octet
(type 'byte')
// Attention : couper ou afficher octet par octet casse les caractères
multi-octets !

// 2. Parcours avec 'for ... range' -> Décode automatiquement les runes UTF-8
(type 'rune' / int32)
for indexOctet, r := range texte {
	fmt.Printf("Octet %d : rune '%c' (Code Unicode: %d / U+%04X)\n", indexOctet,
r, r, r)
}
// Affichera les index d'octets : 0 ('G'), 1 ('o'), 2 (' '), 3 ('🚀').
```

Astuce : Pour obtenir le nombre de caractères (`runes`) sans faire de boucle manuelle, on utilise `utf8.RuneCountInString(texte)` (package `"unicode/utf8"`) ou on convertit en slice de runes `len([]rune(texte))`.

---

3. Constantes et le générateur `iota`

3.1 Constantes non typées (*Untyped Constants*)

Les constantes (`const`) sont évaluées à la compilation et ne peuvent pas être déclarées avec `:=`. Une constante déclarée sans type explicite (ex: `const Max = 1000`) est dite *non typée* : le compilateur la stocke avec une précision haute (au moins 256 bits) et l'adapte automatiquement au type de la variable dans laquelle elle est injectée (`int16`, `int64`, `float64`...).

3.2 Énumérations et Bitflags avec `iota`

Go ne possède pas le mot-clé `enum` du C. À la place, il fournit l'identifiant prédéfini **`iota`** utilisable dans un bloc `const (...)` :

- `iota` commence à `0` à la première ligne du bloc `const`.
- Il s'incrémente de `1` à chaque nouvelle ligne de déclaration dans ce bloc.
- Si une ligne de `const` omet l'expression à droite du `=`, Go **répète automatiquement le type et l'expression de la ligne précédente**.

```go
// 1. Énumération typée classique (Bonne pratique : toujours définir la valeur 0 comme valeur par défaut/invalide)
type NiveauLog uint8

const (
	LogInconnu NiveauLog = iota // 0 (Zero Value sécurisée)
	LogDebug                    // 1
	LogInfo                     // 2
	LogWarning                  // 3
	LogError                    // 4
)

// 2. Masques binaires (Bitflags) avec décalage de bits (1 << iota)
type FlagReseau uint8

const (
	FlagUp        FlagReseau = 1 << iota // 1 << 0 = 1  (00000001)
	FlagBroadcast                        // 1 << 1 = 2  (00000010)
	FlagLoopback                         // 1 << 2 = 4  (00000100)
	_                                    // 1 << 3 = 8  (Ignoré grâce au Blank Identifier '_')
	FlagMulticast                        // 1 << 4 = 16 (00010000)
)
```

---

4. Structures de Contrôle

Go impose un style visuel uniforme : **pas de parenthèses** autour des conditions, **accolades `{}` obligatoires** dans tous les cas, et **aucun opérateur ternaire** (`? :` a été banni par les créateurs de Go car il encourageait des expressions imbriquées illisibles).

4.1 L'instruction `if / else` avec initialisation locale

Une particularité très idiomatique de Go consiste à précéder la condition du `if` par une instruction courte d'initialisation (séparée par un `;`). La variable ainsi créée a une portée (**scope**) strictement restreinte aux blocs `if` et `else` :

```go
// 'latence' est calculée, testée, et détruite dès la sortie du bloc if/else :
if latence := mesurerPing("8.8.8.8"); latence > 100 {
	fmt.Printf("Réseau lent (%d ms)\n", latence)
} else {
	fmt.Printf("Réseau optimal (%d ms)\n", latence)
}
```

```
// Ici, 'latence' n'existe plus : impossible de l'utiliser par erreur dans le
reste de la fonction.
```

4.2 La boucle unique `for` et les Labels de saut

Les mots-clés `while` et `do-while` n'existent pas. Le mot-clé `for` couvre 100 % des besoins :

```
Go
// 1. Style C classique
for i := 0; i < 10; i++ { ... }

// 2. Style 'while' (condition seule)
for tentatives < 3 { ... }

// 3. Boucle infinie (équivalent de 'while(1)' en C, très courant pour les
serveurs et goroutines)
for {
	if arretDemande() {
		break
	}
}
```

Sortir proprement de boucles imbriquées avec les Labels

En C, sortir de deux boucles `for` imbriquées nécessite soit un booléen de contrôle, soit un `goto`. En Go, `break` et `continue` peuvent cibler un **label** placé sur la boucle englobante :

```
Go
RechercheMatrice:
for ligne := 0; ligne < 10; ligne++ {
	for col := 0; col < 10; col++ {
		if estCorrompu(ligne, col) {
			continue RechercheMatrice // Passe immédiatement à la 'ligne'
suivante !
		}
		if cibleTrouvee(ligne, col) {
			break RechercheMatrice // Interrompt instantanément les DEUX
boucles !
		}
	}
}
```

4.3 Le `switch / case` de Go

Le `switch` de Go corrige tous les défauts historiques du `switch` en C :

1. **`break` automatique :** Chaque `case` se termine automatiquement sans tomber dans le cas suivant (si ce comportement rare est souhaité, il faut écrire le mot-clé explicite `fallthrough`).
2. **Valeurs multiples :** On peut regrouper plusieurs cas sur une seule ligne avec des virgules (`case "GET", "HEAD":`).
3. **`switch` sans condition (Switch booléen) :** En omettant la variable après `switch`, chaque `case` peut contenir une expression booléenne complète. C'est la manière officielle en Go de remplacer les longues cascades de `if / else if / else`.

```
Go
caractere := '7'

// Switch sans expression (équivalent d'un if/else if ultra-lisible) :
switch {
case caractere >= '0' && caractere <= '9':
	fmt.Println("C'est un chiffre ASCII")
case caractere >= 'a' && caractere <= 'z', caractere >= 'A' && caractere <= 'Z':
	fmt.Println("C'est une lettre ASCII")
case caractere > 127:
	fmt.Println("Caractère Unicode multi-octets")
default:
	fmt.Println("Symbole ou ponctuation")
}
```